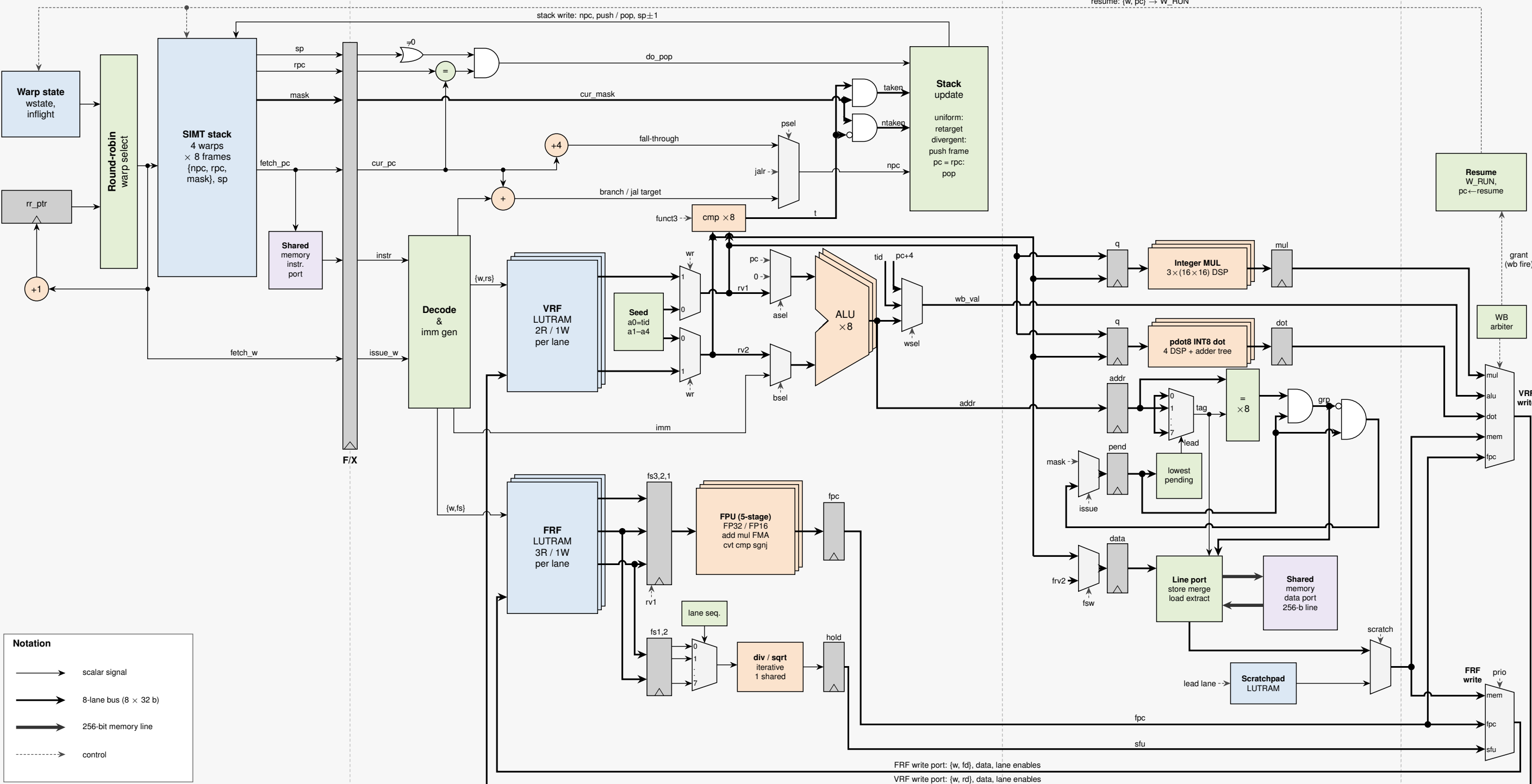




How the datapath operates, step by step

Block names are in **bold**, signal names as **signal**, both exactly as labelled in the figure. The steps follow one instruction from fetch to write-back, then the side engines, memory and write-back.

0 The big picture

SIMTiX runs a *grid* of N threads. Threads are grouped into *warps* of 8; the 8 threads of a warp are the 8 *lanes* and execute the same instruction in lockstep, each on its own data. Four warps are resident at once (4 *warp slots*). All four share one fetch/decode path, one ALU row ($\times 8$ lanes) and one memory port, and take turns cycle by cycle.

The datapath has four regions, left to right:

- **Fetch** picks a warp and reads its next instruction.
- **Issue / execute** decodes it, reads registers, runs the ALU and resolves branches in one cycle.
- **Park-and-resume engines** hold long operations (multiply, dot product, floating point, divide, memory). The warp waits there (*parks*) while the other warps keep issuing: this is how SIMTiX hides latency.
- **Write-back** decides which result writes the register files each cycle and wakes (*resumes*) the warp.

Each warp slot is in one state (**wstate**): W_EMPTY, W_RUN, parked in W_MEM / W_FPC / W_SFU / W_MUL / W_DOT, or W_DONE.

1 Launch: filling the warp slots

The host CPU writes the kernel PC, three base pointers and N through memory-mapped registers and sets GO. The pool latches these values and computes the number of warps, $\lceil N/8 \rceil$. Every cycle in which a slot is free (W_EMPTY or W_DONE) and warps remain, the next warp is placed in that slot:

- its base thread id is warp $\times 8$ (used by **Seed** for a0 = tid);
- the base frame of its **SIMT stack** gets npc = kernel PC, rpc = FFFF_FFFF (never reached, so the base frame is never popped) and a mask of the lanes with tid < N (the last warp may be partial);
- all its “written” bits are cleared, so every register reads its seed value;
- its state becomes W_RUN.

A slot is recycled as soon as its warp retires, so grids larger than 4 warps run without host help.

2 Fetch: choosing a warp

Warp state holds **wstate** and an **inflight** bit per slot. A warp is **runnable** when it is in W_RUN and has no instruction in the X stage. **Round-robin warp select** is a priority encoder that starts its search at slot **rr_ptr** and returns the first runnable warp as **fetch_w**. Starting after the last-fetched warp makes the choice fair: no warp can starve the others. **+1** computes **fetch_w+1**, which becomes the new **rr_ptr** on the clock edge.

3 Fetch: reading the stack and the instruction

fetch_w addresses the **SIMT stack**. The warp’s top frame (index **sp**) supplies:

- **fetch_pc**: the PC of this warp’s next instruction;
- **mask**: which of the 8 lanes are active on this path;
- **rpc**: the join (reconvergence) PC of the current frame;
- **sp**: the frame index itself.

fetch_pc goes to the **shared memory instruction port**, which returns **instr** in the same cycle (asynchronous LUTRAM read).

4 The F/X pipeline register

On the clock edge **F/X** captures **instr**, **fetch_w** (renamed **issue_w**) and the top-frame values (renamed **cur_pc**, **cur_mask**, **cur_rpc**, **cur_sp**). The warp’s **inflight** bit is set. Splitting fetch from execute halves the longest combinational path; the price is that one warp can issue at most every other cycle. With two or more runnable warps, F and X work on different warps and one instruction still issues every cycle. X always *resolves* its instruction in one cycle. It either commits it, parks the warp on an engine, or does nothing (engine busy, or write port lost) so the warp simply fetches the same instruction again. Nothing is committed on a retry, so re-execution is harmless. **inflight** is cleared in every case.

5 Decode and register read

Decode & imm gen splits **instr** into opcode, rd, rs1, rs2, rs3, funct3 and funct7 and builds the sign-extended immediate **imm** (I, S, B, U or J format). It also produces the mux selects (**asel**, **bsel**, **wsel**, **psel**) and the instruction class (ALU, branch, jump, memory, MUL, DOT, FP, div/sqrt, ecall).

The read addresses are {**w,rs**} and {**w,fs**}: warp number (**issue_w**) concatenated with the register number. Each lane has its own LUTRAM bank, so the **VRF** (2 read ports) and **FRF** (3 read ports) deliver the value for all 8 lanes in parallel. FP16 values live NaN-boxed in the low half of FP registers.

6 Seed muxes: registers without a reset

A LUTRAM cannot clear 32 registers in one cycle, so registers are *not* cleared at launch. A per-register **wr** (written) bit is kept instead. Each of the two **seed muxes**, one per read port (rs1 and rs2), passes:

- the VRF value if **wr** = 1 (written since launch);
- otherwise the register’s **Seed**: a0 = thread id (base id + lane), a1–a4 = kernel arguments, anything else 0.

x0 always reads 0. The outputs are the operands **rv1** and **rv2**, 8 lanes wide. The FRF has no seeds: an unwritten FP register reads 0.

7 Operand select and the ALU

Operand A mux (**asel**): **rv1**, **pc** (auipc) or 0 (lui). **Operand B mux** (**bsel**): **rv2** or **imm**. **ALU** $\times 8$ computes add, sub, and, or, xor, shifts and set-less-than in every lane. For loads and stores it computes the effective address, so its result also goes to the memory engine as **addr**.

8 Write-back value

Write-back value mux (**wsel**) chooses what a single-cycle instruction writes:

- the ALU result;
- **pc+4**, the link address of jal / jalr;
- **tid**, for *crr* rd, tid.

The result **wb_val** goes to the **VRF write arbiter**. Lanes that are masked off, and writes to x0, never write.

9 Branch resolution per lane

cmp $\times 8$ evaluates the branch condition selected by **funct3** (beq, bne, blt, bge, bltu, bgeu) on **rv1**, **rv2** in every lane, giving the 8-bit vector **t**.

- **AND** gives **taken** = **t** AND **cur_mask**: active lanes that take the branch.

- **AND with inverted input** gives **ntaken** = **cur_mask** AND NOT **t**: active lanes that fall through.

Inactive lanes are in neither set; they are waiting on another path.

10 Next-PC candidates

- **+4: fall-through** = **cur_pc+4**.
- **+: branch / jal target** = **cur_pc+imm**.
- **jalr**: rv1 of the lowest active lane + imm, bit 0 cleared (jumps through a register are assumed uniform).

Next-PC mux (**psel**) forms **npc** for uniform control flow. It chooses fall-through for ordinary instructions and not-taken branches, the target for taken branches and jal, and the jalr target for jalr.

11 Pop check: reaching the join point

A pushed frame ends when its lanes reach the frame’s join PC.

- **≠0 (OR)**: OR of the 3 bits of **cur_sp**, true when the warp is inside a pushed frame (sp $\neq$ 0).
- **=**: compares **cur_pc** with **cur_rpc**.
- **AND**: both true gives **do_pop**.

When **do_pop** is set the instruction is *not* executed. The frame is popped (sp–1), and the frame below, whose PC is already the join PC and whose mask is the union of both paths, becomes the top. The warp continues there with all its lanes.

12 Stack update

Stack update receives **taken**, **ntaken**, **npc** and **do_pop** and writes the stack (**stack write**). The top frame of warp **issue_w** changes as follows:

- *Ordinary instruction, jump, or uniform branch* (**taken** equals **cur_mask**, or is 0): top npc $\leftarrow$ **npc**.
- *Divergent forward branch* (an if, target > pc): top npc $\leftarrow$ target (the join) with the full mask. A frame is pushed with npc = fall-through, mask = **ntaken**, rpc = target. The fall-through lanes run the if body first, then pop at the join.
- *Divergent backward branch* (a loop): top npc $\leftarrow$ fall-through (the loop exit). A frame is pushed with npc = target, mask = **taken**, rpc = fall-through. The looping lanes run until each exits.
- *Pop*: sp $\leftarrow$ sp–1.

Nesting is allowed up to 8 frames.

13 Example: if (tid & 1) x += 1000

The compiler computes t0 = tid & 1 and emits beqz t0, skip around the body, so the branch is taken by the even lanes. Writing lane masks as lane 7... lane 0, for a full warp t = 01010101, so **taken** = 01010101 and **ntaken** = 10101010: the branch diverges. The top frame becomes {skip, full mask}, and a frame {body, 10101010, rpc = skip} is pushed. The odd lanes execute the body with the even lanes masked off (their writes are suppressed). At skip, pc = rpc, so the pop check fires and the warp continues from skip with all 8 lanes.

14 Parking on an engine

Long operations are not executed in X. If the needed engine is idle, X:

- copies the operands into that engine’s capture register **q**;

- records the destination register, the lane write enables (**cur_mask** and $rd \neq x0$) and the resume PC ($pc+4$);
- sets the warp’s state to the engine’s **W_ state**.

The scheduler then fetches from the other warps. Capture is needed because **rv1**, **rv2** and **frv** follow whichever warp is in X, and change next cycle. If the engine is busy, the warp stays **W_RUN** and retries. Each engine holds one operation at a time.

15 Integer multiply and pdot8

Integer MUL (RV32M **mul**): the low 32 bits of **rv1** × **rv2** per lane. It is built from three 16 × 16 DSP products, $aL \cdot bL + ((aL \cdot bH + aH \cdot bL) \ll 16)$, because the high product $aH \cdot bH$ does not affect the low 32 bits. The pipeline is fully registered and the result arrives 7 cycles after issue. **pdot8 INT8 dot** (custom-0 **pdot8**): each 32-bit operand holds four INT8 values. It computes the sum of the four byte products per lane (signed × signed, unsigned × unsigned, or signed × unsigned, selected by **funct3**) with four DSP multipliers and an adder tree, also in 7 cycles. Both results wait in their hold registers (**mul**, **dot**) for the VRF write port.

16 The FPU

The FRF operands **frv3**, **frv2**, **frv1** (and **rv1** for int-to-float and $fmv.w.x$) are captured. The **FPU (5-stage)** computes FP32 or FP16:

- add, sub, mul, fused multiply-add (one rounding);
- convert, compare, sign-inject, min/max, move, classify.

The result **fpc** is ready 5 cycles later. FP16 is widened to FP32, computed, and rounded back, which gives exactly the FP16 result. **fpc** goes to the FRF, or to the VRF for compares, $fcvt.w$ and $fmv.x.w$.

17 div / sqrt: one shared core

Divide and square root are rare and large, so there is a single core instead of eight.

- The capture register holds **frv1**, **frv2** of all lanes.
- The **lane seq.** (lane sequencer) steps **lane** through the active lanes.
- The **lane mux (0... 7)** feeds that lane’s operands to **div / sqrt**. The core produces one quotient bit per cycle (26 iterations) or one root bit per cycle (37 iterations).
- Each lane’s result is stored in its slot of the **hold** register (**sfu**).

The latency is roughly (number of active lanes) × (26 or 37) cycles. This is acceptable because other warps keep running, and it saves 7 cores’ area and routing.

18 Memory engine: accepting a load or store

One memory instruction can be in flight. At issue the engine captures:

- **addr** of all 8 lanes into **addr**;
- store data into **data**, via the **store-data mux (fsw: rv2 for sw/sh/sb, frv2 for fsw)**;
- **cur_mask** into **pend**, through the **pending mux (issue)**: the lanes still to serve;
- size (**funct3**), destination register, and whether the access is to the scratchpad (**scratch**: address bits 31:30 = 01).

19 Memory engine: coalescing

The memory port moves one whole 32-byte (256-bit) line per cycle. Each cycle:

- **Lowest pending** picks the lowest-numbered pending lane, **lead**.
- The **lead-lane mux (0... 7)** selects that lane’s address; its line tag (bits 31:5) is **tag**.
- **= × 8** compares every lane’s line tag with **tag**, and **AND** with **pend** gives **grp**: all pending lanes on the same line.
- All **grp** lanes are served in this single transaction.
- **AND (inverted input)** computes **pend** AND NOT **grp**. It returns through the pending mux as the next **pend**.

The engine finishes when **pend** becomes 0. A contiguous access ($A[tid]$, 8 words) costs 1 transaction; a fully scattered one costs up to 8.

20 Memory engine: line port, shared memory, scratchpad

Line port, for a store: places each **grp** lane’s data at its word and byte position in a 256-bit line and sets matching byte enables for sb, sh or sw. For a load: extracts each **grp** lane’s word (address bits 4:2), selects byte or halfword (bits 1:0) and sign- or zero-extends it. **Shared memory data port**: the **256-bit line** interface to the 8-bank shared memory, one bank per word of the line, so a whole line is accessed in one cycle. **Scratchpad**: per-warp on-chip RAM (64 words per warp). It is single-ported, so it serves one lane (**lead lane**) per cycle and never touches the shared memory; reused data staged here saves line transactions. **Result mux (scratch)** delivers **mem**, the load result, to write-back. When the last lane is served, the warp is resumed at $pc+4$.

21 Write-back arbitration

Each register file has a single write port per lane, but several producers can finish in the same cycle. **VRF write arbiter**, priority highest first:

1. **mem**: loads, which cannot wait because the line is only present for one cycle;
2. **fpc**: FP results with an integer destination;
3. **mul**;
4. **dot**;
5. **alu (wb_val)**.

FRF write arbiter: **mem** (flw), then **sfu**, then **fpc**. **WB arbiter** computes these selects (**prio**). An engine that loses keeps its result in its hold register and tries again next cycle. A losing single-cycle ALU instruction is *squashed*: nothing is written and the warp re-issues it next cycle, which gives the same result. The winner drives the **VRF write port** or **FRF write port**: {warp, register}, 8 lane values and 8 lane enables (only lanes active at issue write). The register’s **wr** bit is set, so from now on it reads the VRF instead of its seed.

22 Resume

When an engine’s result is granted the port (**grant (wb fire)**), **Resume** writes the warp’s resume PC ($pc+4$ of the parked instruction) into the top frame of its **SIMT stack** and sets its state back to **W_RUN (resume)**. The warp is runnable again from the next cycle. The memory engine resumes its warp in the same way when its last lane has been served.

23 Retire and completion

ecall moves the warp to **W_DONE** and frees its slot for the next warp of the grid. When no warps remain to spawn and no slot is running or parked, the pool pulses *done*. The accelerator then records the kernel’s cycle count and sets STATUS.DONE for the host to read.

24 Latency summary

ALU, branch, jump	1 cycle in X
integer mul, pdot8	7 cycles
FPU	5 cycles after capture
div / sqrt	≈26 / 37 per active lane
load / store	1 cycle per distinct line
scratchpad	1 cycle per active lane

While one warp waits on any of these, the other resident warps keep the fetch and ALU path busy. This overlap of one warp’s wait with other warps’ work is the core idea of a SIMT machine.

Connection reference: every block, its inputs, its outputs

Read a row as “this block receives these signals from these blocks, and sends these signals to these blocks”. *Italic* names are blocks; bold names are signals, as labelled in the figure.

Block	Inputs ← from	Outputs → to	Role in the datapath
Fetch (F)			
<i>Warp state</i>	resume ← <i>Resume</i> ; spawn, park and retire updates from the launch logic and the X stage	runnable → <i>Round-robin warp select</i>	Remembers what every warp slot is doing, so only warps that can make progress are fetched.
<i>rr_ptr</i> <i>+1</i> <i>Round-robin warp select</i> <i>SIMT stack</i>	fetch_w +1 ← <i>+1</i> fetch_w ← <i>Round-robin warp select</i> runnable ← <i>Warp state</i> ; rr_ptr ← <i>rr_ptr</i> fetch_w (read) ← <i>Round-robin warp select</i> ; stack write ← <i>Stack update</i> ; resume ← <i>Resume</i>	rr_ptr → <i>Round-robin warp select</i> → <i>rr_ptr</i> fetch_w → <i>SIMT stack</i> (read address), <i>F/X</i> (becomes issue_w), <i>+1</i> sp , rpc , mask , fetch_pc → <i>F/X</i> ; fetch_pc → <i>Shared memory instr. port</i>	Rotates the search start so every warp gets its turn. Makes the warp after the one just fetched the first candidate next cycle. Chooses the one warp that uses the shared fetch/decode/ALU path in the next cycle. Holds each warp’s control flow: the PC and active-lane mask of every open path, so divergent lanes can run in turn and rejoin.
<i>Shared memory instr. port</i> <i>F/X</i>	fetch_pc ← <i>SIMT stack</i> instr , fetch_w , sp , rpc , mask , fetch_pc from the fetch stage	instr → <i>F/X</i> instr , issue_w → <i>Decode</i> ; cur_sp → <i>≠0</i> ; cur_rpc → <i>=</i> ; cur_pc → <i>=</i> , <i>+</i> , <i>+4</i> ; cur_mask → <i>AND</i> , <i>AND (inverted input)</i> , lane write enables	Supplies the kernel instruction for the selected warp. Pipeline boundary: everything the X stage needs about the fetched warp, frozen for one cycle.
Issue / execute (X): decode, registers, ALU			
<i>Decode & imm gen</i>	instr , issue_w ← <i>F/X</i>	{w,rs} → <i>VRF</i> ; {w,fs} → <i>FRF</i> ; imm → <i>+</i> , <i>Operand B mux</i> ; asel , bsel , wsel , psel , funct3 → muxes and <i>cmp × 8</i> two read values (all 8 lanes) → <i>Seed muxes</i> seed of rs1, seed of rs2 → <i>Seed muxes</i> rv1 , rv2 → <i>Operand A/B muxes</i> , <i>cmp × 8</i> , <i>MUL</i> and <i>pdot8</i> captures; rv2 → <i>Store-data mux</i> ; rv1 → <i>FPU capture</i> → <i>ALU input a</i> → <i>ALU input b</i> result → <i>Write-back value mux</i> ; addr → memory <i>addr</i> register wb_val → <i>VRF write arbiter</i> (input alu)	Turns the instruction bits into register addresses, an immediate, and the control settings of every mux. Integer registers of all threads, one LUTRAM bank per lane. Supplies the launch value of a register not yet written. Produce the true register value without ever clearing the VRF.
<i>VRF</i> <i>Seed</i> <i>Seed muxes</i> (two, one per read port) <i>Operand A mux</i> <i>Operand B mux</i> <i>ALU × 8</i> <i>Write-back value mux</i>	{w,rs} ← <i>Decode</i> ; VRF write port ← <i>VRF write arbiter</i> warp base id, lane number, kernel arguments (latched at launch) <i>VRF data</i> ← <i>VRF</i> ; seed ← <i>Seed</i> ; wr (written bit) rv1 , pc , 0 ; asel ← <i>Decode</i> rv2 , imm ; bsel ← <i>Decode</i> operands ← <i>Operand A/B muxes</i> <i>ALU result</i> , pc+4 , tid ; wsel ← <i>Decode</i>		Selects rs1, the PC (auipc) or zero (lui). Selects rs2 or the immediate. Integer arithmetic for all lanes; also the load/store address generator. Result of any single-cycle instruction.
Issue / execute (X): control flow			
<i>≠0 (OR)</i> <i>= (pop compare)</i> <i>AND (pop)</i> <i>+</i> <i>+4</i> <i>Next-PC mux</i> <i>cmp × 8</i> <i>AND (taken)</i> <i>AND (inverted input)</i> <i>Stack update</i>	cur_sp (3 bits) ← <i>F/X</i> cur_pc , cur_rpc ← <i>F/X</i> outputs of <i>≠0</i> and <i>=</i> cur_pc ← <i>F/X</i> ; imm ← <i>Decode</i> cur_pc ← <i>F/X</i> fall-through , jalr , branch / jal target ; psel rv1 , rv2 ← <i>Seed muxes</i> ; funct3 ← <i>Decode</i> t ; cur_mask ← <i>F/X</i> cur_mask ; t (inverted) taken , ntaken , npc , do_pop	→ <i>AND</i> (pop) → <i>AND</i> (pop) do_pop → <i>Stack update</i> branch / jal target → <i>Next-PC mux</i> fall-through → <i>Next-PC mux</i> npc → <i>Stack update</i> t → <i>AND</i> , <i>AND (inverted input)</i> taken → <i>Stack update</i> ntaken → <i>Stack update</i> stack write → <i>SIMT stack</i> (top channel)	True when the warp is inside a pushed (divergent) frame. True when the path has reached its join PC. Decides to pop the frame instead of executing. PC-relative target. Next sequential PC (also the link value pc+4). The next PC for uniform control flow. Evaluates the branch condition separately in every lane. Active lanes that take the branch. Active lanes that fall through. Retargets, pushes (on divergence) or pops (at the join) the warp’s stack.
Floating point (beside the FRF)			
<i>FRF</i>	{w,fs} ← <i>Decode</i> ; FRF write port ← <i>FRF write arbiter</i>	frv3 , frv2 , frv1 → <i>FPU capture</i> ; frv1 , frv2 → <i>div/sqrt capture</i> ; frv2 → <i>Store-data mux</i> → <i>FPU</i> → <i>fpc</i> hold fpc → <i>FRF write arbiter</i> , <i>VRF write arbiter</i> lanes 0 . . . 7 → <i>Lane mux</i> lane → <i>Lane mux</i> select, hold register slot → <i>div / sqrt</i> → <i>hold</i> sfu → <i>FRF write arbiter</i>	Floating-point registers (FP32, FP16 NaN-boxed), 3 read ports for FMA.
<i>FPU capture q</i> <i>FPU (5-stage)</i> <i>fpc hold</i> <i>div/sqrt capture q</i> <i>lane seq.</i> <i>Lane mux (0 . . . 7)</i> <i>div / sqrt</i> <i>hold</i>	fs3 , fs2 , fs1 ← <i>FRF</i> ; rv1 ← <i>Seed mux</i> captured operands <i>FPU result</i> fs1 , fs2 ← <i>FRF</i> active mask of the parked warp captured lanes; lane one lane’s operands <i>div / sqrt</i> result, slot lane		Freezes the operands of the parked warp. All FP arithmetic except div/sqrt. Keeps the result until a write port is granted. Freezes all lanes’ operands for the sequential core. Walks the active lanes one by one. Feeds one lane at a time into the single core. Iterative divide (26 steps) or square root (37 steps). Collects all lanes before one write-back.

Block	Inputs ← from	Outputs → to	Role in the datapath
Park-and-resume engines			
<i>MUL capture q</i>	rv1 , rv2 (rails)	→ <i>Integer MUL</i>	Freezes the multiply operands.
<i>Integer MUL</i>	captured operands	→ <i>mul</i> hold	32×32 multiply, low word, 7 cycles.
<i>mul hold</i>	product	mul → <i>VRF write arbiter</i>	Waits for the VRF port.
<i>pdot8 capture q</i>	rv1 , rv2 (rails)	→ <i>pdot8 INT8 dot</i>	Freezes the packed INT8 operands.
<i>pdot8 INT8 dot</i>	captured operands	→ <i>dot</i> hold	Sum of four INT8 products, 7 cycles.
<i>dot hold</i>	dot product	dot → <i>VRF write arbiter</i>	Waits for the VRF port.
Coalescing memory engine			
<i>addr register</i>	addr ← <i>ALU</i>	lanes 0 . . . 7 → <i>Lead-lane mux</i> ; all lanes → = × 8	Per-lane addresses of the in-flight access.
<i>Pending mux</i>	mask (cur_mask) at issue ; next pending ← <i>AND (inverted input)</i>	→ <i>pend</i>	Starts with the active lanes, then removes served lanes.
<i>pend register</i>	← <i>Pending mux</i>	pend → <i>Lowest pending, AND</i> → <i>grp, AND (inverted input)</i>	Lanes still to be served.
<i>Lowest pending</i>	pend	lead → <i>Lead-lane mux</i> select	Picks the lane whose line is served this cycle.
<i>Lead-lane mux (0 . . . 7)</i>	lane addresses; lead	tag → = × 8, <i>Line port</i>	The line address of this cycle’s transaction.
<i>= × 8</i>	all lane addresses; tag	→ <i>AND</i> → <i>grp</i>	Finds every lane on the same 32-byte line.
<i>AND</i> → <i>grp</i>	compare results; pend	grp → <i>Line port, AND (inverted input)</i>	Lanes coalesced into this transaction.
<i>AND (inverted input)</i>	grp (inverted); pend	→ <i>Pending mux</i>	Removes the served lanes.
<i>Store-data mux</i>	rv2 , frv2 ; fsw	→ <i>data</i> register	Integer or FP store data.
<i>data register</i>	store data	→ <i>Line port</i>	Per-lane store data of the in-flight access.
<i>Line port</i>	data; tag ; grp ; read line ← <i>Shared memory</i>	256-bit line (address, data, byte enables) → <i>Shared memory</i> ; load data	Merges stores into one line; extracts and sign-extends loads.
		→ <i>Result mux</i>	
<i>Shared memory data port</i>	line address, write data, byte enables ← <i>Line port</i>	read line → <i>Line port</i>	The 8-bank shared memory, one 256-bit line per cycle (outside warp_pool).
<i>Scratchpad</i>	lead lane address and data	read data → <i>Result mux</i>	Per-warp on-chip RAM, one lane per cycle.
<i>Result mux</i>	line-port load data, scratchpad data; scratch	mem → <i>VRF write arbiter, FRF write arbiter</i>	Load result of the access.
Write-back			
<i>VRF write arbiter</i>	mul , alu (wb_val) , dot , mem , fpc ; select ← <i>WB arbiter</i>	VRF write port → <i>VRF</i> (bottom channel)	One integer result per cycle; priority mem, fpc, mul, dot, alu.
<i>FRF write arbiter</i>	mem , fpc , sfu ; prio	FRF write port → <i>FRF</i> (bottom channel)	One FP result per cycle; priority mem, sfu, fpc.
<i>WB arbiter</i>	results waiting in the hold registers	select → <i>VRF write arbiter</i> ; grant (wb fire) → <i>Resume</i>	Grants the port; the loser waits (engine) or re-issues (ALU).
<i>Resume</i>	grant (wb fire) ← <i>WB arbiter</i>	resume → <i>SIMT stack, Warp state</i> (top channel)	Returns the parked warp to W_RUN at its resume PC.

SIMTiX warp_pool datapath: blocks and signals

8 lanes (SIMT threads), 4 resident warps, RV32IMF + Zfh + custom pdot8. “Lane” = one of the 8 threads executed in lockstep; “ $\times 8$ ” marks per-lane replicated hardware; a thick line carries one 32-bit value per lane.

Blocks: fetch		Blocks: floating point (beside the FRF)		Signals: fetch and control		Signals: engines and memory	
Warp state	Per warp slot: <i>wstate</i> (EMPTY, RUN, MEM, SFU, FPC, MUL, DOT, DONE) and <i>inflight</i> (warp has an instruction in the X stage).	FPU (5-stage)	Per-lane pipelined FP32/FP16 unit: add, sub, mul, FMA, convert, compare, sign-inject, min/max, move, classify. Result 5 cycles after issue.	runnable	Warps in W_RUN that are not in flight.	mul, dot	Integer multiply and INT8 dot-product results.
rr_ptr	Round-robin pointer: the warp slot where the next fetch search starts.	lane seq.	Lane sequencer: steps through the active lanes one at a time for the shared div/sqrt.	fetch_w	Warp chosen for fetch this cycle.	fpc	FPU result (to the FRF, or the VRF for compares, fcvt.w and fmv.x.w).
+1	Increments the fetched warp id to form the next <i>rr_ptr</i> .	Lane mux (0... 7)	8:1 mux that feeds the selected lane's captured operands to div/sqrt.	sp, rpc, mask	Top-of-stack pointer, reconvergence PC and active-lane mask of the fetched warp.	sfu, hold	div / sqrt results, collected lane by lane.
Round-robin warp select	Priority encoder: the first warp from <i>rr_ptr</i> that is in W_RUN and not in flight.			fetch_pc	Top-of-stack next PC: the instruction address.	lane	Lane currently in the shared div/sqrt core.
SIMT stack	Per-warp reconvergence stack, 8 frames of {npc, rpc, mask} plus stack pointer <i>sp</i> . The top frame gives the warp's PC and active mask.	div / sqrt	One shared iterative divide / square-root core, about 26 (div) or 37 (sqrt) cycles per lane.	instr	Fetched 32-bit instruction.	mask, issue	<i>cur_mask</i> loaded into the pending register when a memory instruction issues.
Shared memory instr. port	Read port of the on-chip shared memory that returns the kernel instruction at <i>fetch_pc</i> (outside warp_pool).	Blocks: park-and-resume engines		issue_w	Warp executing in the X stage.	pend	Lanes of the in-flight memory instruction not yet served.
F/X	Pipeline register between fetch and execute: <i>instr</i> , <i>issue_w</i> and the top-of-stack context.	q (capture register)	Operands latched when the instruction issues and held while its warp is parked.	cur_pc, cur_rpc, cur_sp, cur_mask	The X-stage copies of pc, rpc, sp and mask.	lead	Lowest pending lane: its line is served this cycle.
Blocks: issue / execute		mul, dot, fpc, hold	Result hold registers: keep a finished result until its write port is granted.	do_pop		tag	Lead lane's line address (address bits 31:5).
Decode & imm gen	Decodes the instruction, forms register-file read addreeses and the immediate.	Integer MUL	RV32M <i>mul1</i> (low 32 bits) from three 16×16 DSP products, 7 cycles.	t		grp	Lanes on the lead line: one coalesced transaction.
VRF	Integer vector register file: one LUTRAM bank per lane, 4 warps × 32 registers, 2 read and 1 write port.	pdot8 INT8 dot	Custom-0 instruction: 4-way INT8 dot product, four DSP products and an adder tree.	taken		fsw, frv2	Store-data select and FP store data.
FRF	Floating-point register file, same organisation, 3 read ports (FMA). FP16 values are NaN-boxed.	Blocks: coalescing memory engine		ntaken	Top frame has reached its join: pop it.	256-bit line	Line address, write data and byte enables to, and read line from, shared memory.
Seed	Launch values: a0 = thread id, a1–a4 = kernel arguments, others 0.	addr, data, pend	Latched per-lane address, per-lane store data and pending-lane mask of the one memory instruction in flight.	func3	Per-lane branch condition from the comparators.	lead lane	Scratchpad address and data from the lead pending lane.
Seed mux (wr)	Reads the RAM if the register was written in this grid, else its seed value.	Pending mux (issue)	Loads <i>cur_mask</i> at issue, afterwards pending AND NOT grp.	fall-through	Active lanes whose branch is taken (t AND <i>cur_mask</i>).	scratch	The access is in the scratchpad aperture (address bits 31:30 = 01).
Operand A mux (asel)	rv1, pc (auipc) or 0 (lui).	lowest pending	Priority encoder: the lowest-numbered pending lane (the lead lane).	branch / jal target	Active lanes that fall through (<i>cur_mask</i> AND NOT t). Instruction field that selects the comparison.	mem	Load result, per lane.
Operand B mux (bsel)	rv2 or the immediate.	Lead-lane mux (0... 7)	8:1 mux that selects the lead lane's address. Its line tag is the line fetched this cycle.	jalr	pc + 4.	Signals: writeback	
ALU ×8	One 32-bit ALU per lane. Its result is also the load/store address.	= ×8 (tag compare)	Per-lane comparator: does this lane's 32-byte line tag equal the lead tag?	npc	pc + immediate.	mul, alu, dot, mem, fpc	VRF write arbiter inputs, highest priority first: mem, fpc, mul, dot, alu.
Write-back value mux (wsel)	ALU result, pc+4 (jal/jalr link) or tid (csrr).	AND → grp	Lanes both pending and on the lead line: served together in one transaction.	psel	jalr target: rv1 of the lead lane + imm, bit 0 cleared.	mem, fpc, sfu	FRF write arbiter inputs, highest priority first: mem, sfu, fpc.
≠0 (OR)	OR-reduction: <i>cur_sp</i> is non-zero (inside a pushed frame).	AND (inverted input)	Next pending mask = pending AND NOT grp.	stack write	Next PC chosen by the next-PC mux.	prio	Write-port priority select.
= (pop check)	<i>cur_pc</i> equals <i>cur_rpc</i> (reached the join point).	Store-data mux (fsw)	rv2 for sw/sh/sb, frv2 for fsw.	resume	Next-PC select, from decode and the branch outcome.	grant (wb fire)	An engine's result took the write port this cycle.
+	Branch / jal target: <i>cur_pc</i> + imm.	Line port	Merges the grp lanes' store data into one 256-bit line with byte enables. Extracts and sign-extends each lane's load.		Update of the SIMT stack: new npc, push or pop, sp ± 1.	VRF / FRF write port	{warp, destination register}, per-lane data and per-lane write enables (masked-off lanes never write).
+4	Fall-through PC.	Shared memory data port	256-bit line port of the 8-bank shared memory (outside warp_pool).	imm	{warp, resume PC} of a warp whose engine result was written back. Sets W_RUN.	Warp states (wstate)	
Next-PC mux (psel)	Fall-through, jalr target or branch / jal target.	Scratchpad	Per-warp on-chip LUTRAM, 64 words per warp at 0x4000_0000, served one lane per cycle.	wr		W_RUN	Runnable.
cmp ×8	Per-lane branch comparator (beq, bne, blt, bge, bltu, bgeu).	Result mux (scratch)	Load data from the line port or from the scratchpad.	rv1, rv2		W_MEM	Parked on the memory engine.
Stack update	Uniform branch: retarget the top frame. Divergent: push a frame so the two lane groups run in turn. pc = rpc: pop and reconverge.	Blocks: writeback		frv1–3, fs1–3		W_FPC, W_SFU	Parked on the FPU or div/sqrt.
		VRF write arbiter	Priority mux onto the single VRF write port: mem, fpc (integer destination), mul, dot, ALU. A losing ALU write is squashed and the instruction re-issues.	pc, 0		W_MUL, W_DOT	Parked on the multiplier or dot engine.
		FRF write arbiter	Priority mux onto the FRF write port: mem (flw), sfu, fpc.	asel, bsel, wsel		W_EMPTY, W_DONE	Free slot, or retired by eca11 (slot can be refilled).
		WB arbiter	Computes the arbiter selects and grants the port to one engine.	pc+4, tid			
		Resume	A granted engine's warp returns to W_RUN with its resume PC written to the stack.	wb_val			
				addr			
				rv1 (FPU)			